

Assignment 2

COMP9021, Trimester 1, 2026

1 General matters

1.1 Aim

The purpose of the assignment is to:

- develop your understanding of how file differences can be represented and processed;
- practice interpreting and validating structured text input;
- design and implement classes with clearly defined responsibilities;
- reinforce your understanding of sequences, subsequences, and basic algorithmic ideas;
- practice working with strings, files, and exceptions in Python;
- produce precise and well-formatted output.

1.2 Submission

Your program should be stored in a file named `diff.py`, optionally together with additional files. After developing and testing your program, upload it via Ed (unless you worked directly in Ed). Assignments can be submitted multiple times; only the last submission will be graded. Your assignment is due on April 27 at 11:59am.

1.3 Assessment

The assignment is worth 13 marks and will be tested against multiple inputs. For each test, the automarking script allows your program to run for 30 seconds.

Assignments may be submitted up to 5 days after the deadline. The maximum mark decreases by 5% for each full late day, up to a maximum of five days. For example, if students *A* and *B* submit assignments originally worth 12 and 11 marks, respectively, two days late (i.e., more than 24 hours but no more than 48 hours late), the maximum mark obtainable is 11.7. Therefore, *A* receives $\min(11.7, 12) = 11.7$ and *B* receives $\min(11.7, 11) = 11$.

1.4 Reminder on plagiarism policy

You are encouraged to discuss strategies for solving the assignment with others; however, discussions must focus on algorithms, not code. You must implement your solution independently. Submissions are routinely scanned for similarities that arise from copying, modifying others' work, or collaborating too closely on a single implementation. Severe penalties apply.

2 How diff Works: Commands and Longest Common Subsequences

This assignment uses several text files that are provided alongside this PDF.

Consider the two files `file_1_1.txt` and `file_1_2.txt`, shown below with line numbers:

```
$ cat -n file_1_1.txt
```

```
1      A line to delete: 1
2      A line to delete: 2
3      A line that stays: 1
4      A line that stays: 2
5      A line to change: 1
6      A line that stays: 3
7      A line that stays: 4
8      A line that stays: 5
9      A line that stays: 6
10     A line to delete: 3
11     A line that stays: 7
12     A line that stays: 8
13     A line to change: 2
14     A line to change: 3
15     A line to change: 4
16     A line to change: 5
17     A line that stays: 9
```

```
$ cat -n file_1_2.txt
```

```
1      A line that stays: 1
2      A line to insert: 1
3      A line that stays: 2
4      A changed line: 1
5      A line that stays: 3
6      A line that stays: 4
7      A line to insert: 2
8      A line to insert: 3
9      A line that stays: 5
10     A line that stays: 6
11     A line that stays: 7
12     A line that stays: 8
13     A changed line: 2
14     A changed line: 3
15     A changed line: 4
16     A line that stays: 9
```

The `diff` command can be used to compare `file_1_1.txt` and `file_1_2.txt`. Its output for these two files is shown below.

```
$ diff file_1_1.txt file_1_2.txt
1,2d0
< A line to delete: 1
< A line to delete: 2
3a2
> A line to insert: 1
5c4
< A line to change: 1
---
> A changed line: 1
7a7,8
> A line to insert: 2
> A line to insert: 3
10d10
< A line to delete: 3
13,16c13,15
< A line to change: 2
< A line to change: 3
< A line to change: 4
< A line to change: 5
---
> A changed line: 2
> A changed line: 3
> A changed line: 4
```

Study this output carefully to understand how `diff` represents differences between two files.

- **d** (*delete*): indicates that lines should be removed from the first file.
 - Form: `l1 d l2`: delete line `l1` from the first file; this deletion occurs after line `l2` in the second file.
 - Form: `l1, l2 d l3`: delete lines `l1` to `l2` from the first file; this deletion occurs after line `l3` in the second file.
- **a** (*add*): indicates that lines are added to the first file.
 - Form: `l1 a l2`: add line `l2` from the second file after line `l1` in the first file.
 - Form: `l1 a l2, l3`: add lines `l2` to `l3` from the second file after line `l1` in the first file.
- **c** (*change*): indicates that lines in the first file are replaced.
 - Form: `l1 c l2`: replace line `l1` in the first file with line `l2` from the second file.
 - Form: `l1, l2 c l3`: replace lines `l1` to `l2` in the first file with line `l3` from the second file.
 - Form: `l1 c l2, l3`: replace line `l1` in the first file with lines `l2` to `l3` from the second file.
 - Form: `l1, l2 c l3, l4`: replace lines `l1` to `l2` in the first file with lines `l3` to `l4` from the second file.

The `diff` command works by identifying a *longest common subsequence* (LCS) of lines shared by both files. This is the largest sequence of lines that appear in the same order in both files (not necessarily consecutively). Based on this subsequence, `diff` computes a minimal set of editing commands that transform the first file into the second. These commands correspond directly to operations supported by the `ed` editor. In the example above, there is a unique longest common subsequence, consisting of the following lines:

```
A line that stays: 1
A line that stays: 2
...
A line that stays: 8
A line that stays: 9
```

The longest common subsequence (LCS) is not always unique.

Consider the following two files:

```
$ cat -n file_2_1.txt
   1      A line
$ cat -n file_2_2.txt
   1      A line
   2      A line
```

In this case, there are two possible LCSs. The single line from `file_2_1.txt` can match either the first or the second line in `file_2_2.txt`.

The `diff` command selects one of these possible LCSs and produces the corresponding sequence of editing commands. For example, it may produce:

```
$ diff file_2_1.txt file_2_2.txt
1a2
> A line
```

This output means that the second line of `file_2_2.txt` is added after line 1 of `file_2_1.txt`.

Another valid implementation could choose the alternative LCS and produce:

```
$ diff file_2_1.txt file_2_2.txt
0a1
> A line
```

This output instead inserts a line at the beginning of the file.

Consider another example with the files `file_3_1.txt` and `file_3_2.txt`.

```
$ cat -n file_3_1.txt
 1      Line 1
 2      Line 2
 3      A line to go
 4      A line to go
 5      A line to go
 6      Line 3
 7      Line 4
 8      A line to go
$ cat -n file_3_2.txt
 1      A line to come
 2      A line to come
 3      Line 1
 4      Line 2
 5      A line to come
 6      A line to come
 7      Line 1
 8      Line 2
 9      A line to come
10      A line to come
11      A line to come
12      Line 3
13      Line 4
```

In this case too, the longest common subsequence (LCS) is not unique. The lines `Line 1` and `Line 2` each appear once in the first file, but twice in the second file. As a result, there are multiple valid ways to match them while preserving order.

In fact, there are three possible LCSs:

- Match both lines with their first occurrences in the second file.
- Match Line 1 with its first occurrence and Line 2 with its second occurrence.
- Match both lines with their second occurrences in the second file.

Each of these choices produces a different valid LCS, and therefore a different valid sequence of editing commands.

The `diff` command selects one of these LCSs and outputs the corresponding commands. For example:

```
$ diff file_3_1.txt file_3_2.txt
0a1,2
> A line to come
> A line to come
3,5c5,11
< A line to go
< A line to go
< A line to go
---
> A line to come
> A line to come
> Line 1
> Line 2
> A line to come
> A line to come
> A line to come
8d13
< A line to go
```

However, other implementations (or different choices of LCS) could produce different, but still valid, outputs. For instance:

- Alternative 1:

```
0a1,2
> A line to come
> A line to come
1a4,7
> Line 2
> A line to come
> A line to come
> Line 1
3,5c9,11
< A line to go
< A line to go
< A line to go
---
> A line to come
> A line to come
> A line to come
8d13
< A line to go
```

- Alternative 2:

```
0a1,6
> A line to come
> A line to come
> Line 1
> Line 2
> A line to come
> A line to come
3,5c9,11
< A line to go
< A line to go
< A line to go
---
> A line to come
> A line to come
> A line to come
8d13
< A line to go
```

One may instead wish to display the longest common subsequence (LCS) itself. To do this, we list the lines that belong to the LCS, and use `...` to represent maximal nonempty sequences of consecutive lines that are not part of the LCS. This can be done with respect to either file:

- with respect to the first file: we show the LCS as it appears in the first file;
- with respect to the second file: we show the LCS as it appears in the second file.

For the `diff` between `file_1_1.txt` and `file_1_2.txt`, this gives the following.

- With respect to `file_1_1.txt`:

```
...
A line that stays: 1
A line that stays: 2
...
A line that stays: 3
A line that stays: 4
A line that stays: 5
A line that stays: 6
...
A line that stays: 7
A line that stays: 8
...
A line that stays: 9
```

- With respect to `file_1_2.txt`:

```
A line that stays: 1
...
A line that stays: 2
...
A line that stays: 3
A line that stays: 4
...
A line that stays: 5
A line that stays: 6
A line that stays: 7
A line that stays: 8
...
A line that stays: 9
```


For the `diff` between `file_2_1.txt` and `file_2_2.txt`, this gives the following.

- With respect to `file_2_1.txt`:

A line

- With respect to `file_2_2.txt`:

A line

...

For the `diff` between `file_3_1.txt` and `file_3_2.txt`, this gives the following.

- With respect to `file_3_1.txt`:

Line 1

Line 2

...

Line 3

Line 4

...

- With respect to `file_3_2.txt`:

...

Line 1

Line 2

...

Line 3

Line 4

3 Task specifications

Write a program in a file named `diff.py`. Your program must define the following three classes:

- `DiffCommands`
- `DiffCommandsError` (which must inherit from `Exception`)
- `OriginalNewFiles`

In the following, the first file (also called the original file) is the first argument passed to `OriginalNewFiles`, and the second file (also called the new file) is the second argument.

3.1 Class `DiffCommands`

This class represents a sequence of `diff` commands.

- It provides no public methods.
- Its constructor (`__init__`) takes as input a text file that is expected to contain a valid sequence of commands that could be produced by `diff` for some pair of files. The file must also satisfy the following formatting constraints:
 - one command per line;
 - no spaces within a line;
 - no extra or empty lines.

If the file does not satisfy these conditions, the constructor must raise a `DiffCommandsError` (1 mark) with the message:

`Cannot possibly be the commands for the diff of two files`

Seven invalid examples are provided (`wrong_1.txt` to `wrong_7.txt`).

Three valid examples are also provided:

- `diff_1.txt`: the unique sequence of commands for `file_1_1.txt` and `file_1_2.txt`
- `diff_2.txt`: one of the two possible sequences for `file_2_1.txt` and `file_2_2.txt`
- `diff_3.txt`: one of the three possible sequences for `file_3_1.txt` and `file_3_2.txt`

The class must implement the `__str__()` method (3 marks), so that calling `print()` on a `DiffCommands` object outputs its commands.

3.2 Class `OriginalNewFiles`

This class provides methods to analyse and display differences between two files. It must implement the following methods:

- `is_valid_diff()` (2 marks), which returns a Boolean;
- `print_diff()`, which prints the corresponding diff output (2 marks);
- `print_unmodified_from_original()` (1.5 marks), which prints the lines of the longest common subsequence as they appear in the original file, with `...` standing for maximal nonempty blocks of omitted lines;
- `print_unmodified_from_new()` (1.5 marks), which prints the lines of the longest common subsequence as they appear in the new file, with `...` standing for maximal nonempty blocks of omitted lines;
- `all_diff_commands()` (2 marks), which returns the list of all possible `diff` command sequences.

If the two files are identical, then `all_diff_commands()` returns a list containing a single `DiffCommands` object whose sequence of commands is empty.

3.3 Expected behaviour

An example interaction (provided below) illustrates:

- how these methods should be called;
- whether they return a value or print their result.

Methods whose names start with `print_` print their output directly and do not return a value. One additional requirement is not immediately visible in the example:

- The list returned by `OriginalNewFiles.pair_of_files.all_diff_commands()` must be sorted lexicographically according to the string representation of its elements.

```

$ python3
...
>>> from diff import *
>>> DiffCommands('wrong_1.txt')
...
diff.DiffCommandsError: Cannot possibly be the commands for the diff of two files
>>> DiffCommands('wrong_2.txt')
...
diff.DiffCommandsError: Cannot possibly be the commands for the diff of two files
>>> DiffCommands('wrong_3.txt')
...
diff.DiffCommandsError: Cannot possibly be the commands for the diff of two files
>>> DiffCommands('wrong_4.txt')
...
diff.DiffCommandsError: Cannot possibly be the commands for the diff of two files
>>> DiffCommands('wrong_5.txt')
...
diff.DiffCommandsError: Cannot possibly be the commands for the diff of two files
>>> DiffCommands('wrong_6.txt')
...
diff.DiffCommandsError: Cannot possibly be the commands for the diff of two files
>>> DiffCommands('wrong_7.txt')
...
diff.DiffCommandsError: Cannot possibly be the commands for the diff of two files
>>> DiffCommands('diff_1.txt')
<diff.DiffCommands object at ...>

```

```

>>> diff_1 = DiffCommands('diff_1.txt')
>>> print(diff_1)
1,2d0
3a2
5c4
7a7,8
10d10
13,16c13,15
>>> diff_2 = DiffCommands('diff_2.txt')
>>> print(diff_2)
1a2
>>> diff_3 = DiffCommands('diff_3.txt')
>>> print(diff_3)
0a1,2
3,5c5,11
8d13
>>> pair_of_files = OriginalNewFiles('file_1_1.txt', 'file_1_2.txt')
>>> pair_of_files.is_valid_diff(diff_1)
True
>>> pair_of_files.is_valid_diff(diff_2)
False
>>> pair_of_files.is_valid_diff(diff_3)
False
>>> pair_of_files.print_diff(diff_1)
1,2d0
< A line to delete: 1
< A line to delete: 2
3a2
> A line to insert: 1
5c4
< A line to change: 1
---
> A changed line: 1
7a7,8
> A line to insert: 2
> A line to insert: 3
10d10
< A line to delete: 3
13,16c13,15
< A line to change: 2
< A line to change: 3
< A line to change: 4
< A line to change: 5
---
> A changed line: 2
> A changed line: 3
> A changed line: 4

```

```

>>> pair_of_files.print_unmodified_from_original(diff_1)
...
A line that stays: 1
A line that stays: 2
...
A line that stays: 3
A line that stays: 4
A line that stays: 5
A line that stays: 6
...
A line that stays: 7
A line that stays: 8
...
A line that stays: 9
>>> pair_of_files.print_unmodified_from_new(diff_1)
A line that stays: 1
...
A line that stays: 2
...
A line that stays: 3
A line that stays: 4
...
A line that stays: 5
A line that stays: 6
A line that stays: 7
A line that stays: 8
...
A line that stays: 9
>>> pair_of_files.all_diff_commands()
[<diff.DiffCommands object at ...>]
>>> diffs = pair_of_files.all_diff_commands()
>>> len(diffs)
1
>>> print(diffs[0])
1,2d0
3a2
5c4
7a7,8
10d10
13,16c13,15
>>> pair_of_files = OriginalNewFiles('file_1_2.txt', 'file_1_1.txt')
>>> diffs = pair_of_files.all_diff_commands()
>>> len(diffs)
1
>>> print(diffs[0])
0a1,2
2d3
4c5
7,8d7
10a10
13,15c13,16

```

```

>>> pair_of_files = OriginalNewFiles('file_1_1.txt', 'file_1_1.txt')
>>> diffs = pair_of_files.all_diff_commands()
>>> len(diffs)
1
>>> print(diffs[0])

>>> pair_of_files = OriginalNewFiles('file_2_1.txt', 'file_2_2.txt')
>>> pair_of_files.is_valid_diff(diff_1)
False
>>> pair_of_files.is_valid_diff(diff_2)
True
>>> pair_of_files.is_valid_diff(diff_3)
False
>>> pair_of_files.print_diff(diff_2)
1a2
> A line
>>> pair_of_files.print_unmodified_from_original(diff_2)
A line
>>> pair_of_files.print_unmodified_from_new(diff_2)
A line
...
>>> diffs = pair_of_files.all_diff_commands()
>>> len(diffs)
2
>>> print(diffs[0])
0a1
>>> print(diffs[1])
1a2
>>> pair_of_files = OriginalNewFiles('file_2_2.txt', 'file_2_1.txt')
>>> diffs = pair_of_files.all_diff_commands()
>>> len(diffs)
2
>>> print(diffs[0])
1d0
>>> print(diffs[1])
2d1
>>> pair_of_files = OriginalNewFiles('file_3_1.txt', 'file_3_2.txt')
>>> pair_of_files.is_valid_diff(diff_1)
False
>>> pair_of_files.is_valid_diff(diff_2)
False
>>> pair_of_files.is_valid_diff(diff_3)
True

```

```

>>> pair_of_files.print_diff(diff_3)
0a1,2
> A line to come
> A line to come
3,5c5,11
< A line to go
< A line to go
< A line to go
---
> A line to come
> A line to come
> Line 1
> Line 2
> A line to come
> A line to come
> A line to come
8d13
< A line to go
>>> pair_of_files.print_unmodified_from_original(diff_3)
Line 1
Line 2
...
Line 3
Line 4
...
>>> pair_of_files.print_unmodified_from_new(diff_3)
...
Line 1
Line 2
...
Line 3
Line 4
>>> diffs = pair_of_files.all_diff_commands()
>>> len(diffs)
3
>>> print(diffs[0])
0a1,2
1a4,7
3,5c9,11
8d13
>>> print(diffs[1])
0a1,2
3,5c5,11
8d13
>>> print(diffs[2])
0a1,6
3,5c9,11
8d13
>>> pair_of_files = OriginalNewFiles('file_3_2.txt', 'file_3_1.txt')
>>> diffs = pair_of_files.all_diff_commands()
>>> len(diffs)
3

```



```
>>> print(diffs[0])
1,2d0
4,7d1
9,11c3,5
13a8
>>> print(diffs[1])
1,2d0
5,11c3,5
13a8
>>> print(diffs[2])
1,6d0
9,11c3,5
13a8
```